```
# CHAPTER 4 Decision Making for a Single Sample

# § 4-1 Statistical Inference

# Figure 4-1 Relationship between a population and a sample.

# § 4-2 Point Estimation

# EXAMPLE 4-1 Unbiased Estimators

# Figure 4-2 The sampling distributions of two unbiased estimators Θ̂₁ and
# Θ̂₂.

# --- 1. Setup ---
if (!require("pacman")) install.packages("pacman")
pacman::p_load(tidyverse, patchwork, pwr, car)

# --- 2. Data Preparation ---
tc_values <- c(41.60, 41.48, 42.34, 41.95, 41.86,
               42.18, 41.72, 42.26, 41.81, 42.04)

armco_df <- data.frame(
  TC = tc_values,
  ObsNo = seq_along(tc_values)
)

# --- 3. Summary Statistics Function ---
get_stats <- function(x) {
  data.frame(
    n          = length(x),
    Mean       = mean(x),
    Var        = var(x),
    Std_Dev    = sd(x),
    Std_Error  = sd(x) / sqrt(length(x)),
    Median     = median(x)
  ) %>% mutate(across(everything(), ~round(., 4)))
}
```

```
cat("\n--- 1. Summary Statistics ---\n")
print(get_stats(armco_df$TC))

# --- 4. Visual Diagnostics ---
p1 <- ggplot(armco_df, aes(x = TC)) +
  geom_histogram(aes(y = after_stat(density)), bins = 5,
                 fill = "steelblue", color = "white") +
  stat_function(fun = dnorm,
                args = list(mean = mean(tc_values), sd = sd(tc_values)),
                color = "firebrick", linewidth = 1) +
  labs(title = "Density vs. Normal", x = "TC Value")

p2 <- ggplot(armco_df, aes(sample = TC)) +
  stat_qq() + stat_qq_line(color = "firebrick") +
  labs(title = "Normal Q-Q Plot", x = "Theoretical", y = "Sample")

p3 <- ggplot(armco_df, aes(x = ObsNo, y = TC)) +
  geom_line(color = "gray70") +
  geom_point(size = 2, color = "darkgreen") +
  geom_hline(yintercept = mean(tc_values), linetype = "dashed",
             color = "blue") +
  labs(title = "Run Chart", x = "Observation #", y = "TC")

p4 <- ggplot(armco_df, aes(x = "", y = TC)) +
  geom_boxplot(fill = "gray90", outlier.color = "red") +
  geom_jitter(width = 0.05, color = "red", alpha = 0.5) +
  coord_flip() + labs(title = "Boxplot", x = "", y = "TC")

combined_plot <- (p1 | p2) / (p3 | p4) +
  plot_annotation(title = "Diagnostic Plots for TC Values",
                  subtitle = "Single Sample Analysis")
print(combined_plot)

# --- 5. Statistical Inference & Power ---
mu_null <- 42.0
alpha   <- 0.05
norm_test <- shapiro.test(tc_values) # Fixed: Defined norm_test
```

```
cat("\n--- 2. T-Test & Normality Results ---\n")
cat("Shapiro-Wilk P-value:", round(norm_test$p.value, 4), "\n")

alts <- c("two.sided", "greater", "less")
t_results <- map_df(alts, function(alt) {
  res <- t.test(armco_df$TC, mu = mu_null, alternative = alt)

  # Effect size (Cohen's d)
  d_val <- (mean(tc_values) - mu_null) / sd(tc_values)

  pwr_val <- pwr.t.test(n = length(tc_values), d = d_val,
                        sig.level = alpha, type = "one.sample",
                        alternative = alt)$power

  data.frame(Alternative = alt, P_Val = round(res$p.value, 4),
             Lower = round(res$conf.int[1], 4),
             Upper = round(res$conf.int[2], 4),
             Power = round(pwr_val, 4))
})
print(t_results)

# --- 6. Custom Inference Function ---
run_inference <- function(data, mu_null, conf_level = 0.95) {
  t_res <- t.test(data, mu = mu_null, conf.level = conf_level)
  w_res <- wilcox.test(data, mu = mu_null, conf.level = conf_level,
conf.int = TRUE, exact = FALSE)

  cat(sprintf("\n--- Inference Results (%d%% Confidence) ---\n",
              round(conf_level * 100)))
  cat("T-Test (Mean)   P:", round(t_res$p.value, 4), "| CI: [",
      round(t_res$conf.int[1], 4), ",", round(t_res$conf.int[2], 4), "]\n")
  cat("Wilcox (Median) P:", round(w_res$p.value, 4), "| CI: [",
      round(w_res$conf.int[1], 4), ",", round(w_res$conf.int[2], 4), "]\n")
}

run_inference(tc_values, mu_null, 0.95)
```

```
# --- 7. Power Analysis (Parametric) ---
cat("\n--- 3. Power Analysis (Alpha = 0.05) ---\n")
d_effect <- abs(mean(tc_values) - mu_null) / sd(tc_values)

power_stats <- lapply(alts, function(alt) {
  d_signed <- if(alt == "less") -d_effect else d_effect
  pwr_res <- pwr.t.test(n = length(tc_values), d = d_signed,
                        sig.level = alpha, type = "one.sample",
                        alternative = alt)
  req_n <- pwr.t.test(d = d_signed, power = 0.80, sig.level = alpha,
                      type = "one.sample", alternative = alt)$n
  data.frame(Alt = alt, Power = round(pwr_res$power, 4),
             Req.N = ceiling(req_n))
})
print(do.call(rbind, power_stats))

# --- 8. Resampling & Final Report ---
generate_final_report <-
 function(data, mu_null, alpha = 0.05, n_sim = 10000) {
  set.seed(123)

  # Permutation Test (Two-sided)
  observed_diff <- mean(data) - mu_null
  centered_data <- data - mean(data) + mu_null
  perm_diffs <- replicate(n_sim,
    mean(sample(centered_data, replace = TRUE)) - mu_null)
  perm_p <- mean(abs(perm_diffs) >= abs(observed_diff))

  # Bootstrap CI
  boot_means <- replicate(n_sim, mean(sample(data, replace = TRUE)))
  boot_ci <- quantile(boot_means, probs = c(alpha/2, 1 - alpha/2))
```

```
  cat("\n==================================================\n")
  cat("   FINAL STATISTICAL REPORT (H0: mu =", mu_null, ")\n")
  cat("==================================================\n")
  cat(sprintf("Observed Mean:    %.4f\n", mean(data)))
  cat(sprintf("Bootstrap 95%% CI: [%.4f, %.4f]\n", boot_ci[1], boot_ci[2]))
  cat(sprintf("Permutation P:    %.4f\n", perm_p))
  cat("Decision:",
      if(perm_p < alpha) "REJECT H0" else "FAIL TO REJECT H0", "\n")
  cat("==================================================\n")

  # Plot Bootstrap Distribution
  ggplot(data.frame(Mean = boot_means), aes(x = Mean)) +
    geom_density(fill = "skyblue", alpha = 0.4) +
    geom_vline(xintercept = mu_null, color = "red", linetype = "dashed",
               linewidth = 1) +
    geom_vline(xintercept = mean(data), color = "darkgreen",
               linewidth = 1) +
    labs(title = "Bootstrap Distribution of Mean",
         subtitle = "Red = Null Hypo, Green = Sample Mean")
}

# Run final analysis
generate_final_report(tc_values, mu_null = 42.0)

# EXERCISES FOR § 4-2
```

§ 4-3 Hypothesis Testing

§ 4-3.1 Statistical Hypotheses

§ 4-3.2 Testing Statistical Hypotheses

Figure 4-3 Decision criteria for testing H0: μ = 50 cm/s versus H1: μ
$\neq$ 50 cm/s.

Table 4-1 Decisions in Hypothesis Testing

Figure 4-4 The critical region for H0: μ = 50 versus H1: μ $\neq$ 50 and n
= 10.

Figure 4-5 The probability of type II error when μ = 52 and n = 10.

Figure 4-6 The probability of type II error when μ = 50.5 and n = 10.

Figure 4-7 The probability of type II error when μ = 52 and n = 16.

§ 4-3.3 P-Values in Hypothesis Testing

Figure 4-8 Calculating the p-value for the propellant burning rate
problem.

§ 4-3.4 One-Sided and Two-Sided Hypotheses

EXAMPLE 4-2 The Rocket Propellant

§ 4-3.5 General Procedure for Hypothesis Testing

EXERCISES FOR § 4-3

```
# § 4-4 Inference on the Mean of a Population, Variance Known

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(HH, BSDA, ggplot2, purrr, dplyr)

# --- 2. Functions ---
Pwr.z.test <- function(n, delta, sigma, alpha, alt = "two.sided") {
  m <- delta / (sigma / sqrt(n))
  if (alt == "two.sided") {
    return(1 - pnorm(qnorm(1 - alpha/2) - m) + pnorm(qnorm(alpha/2) - m))
  } else if (alt == "greater") {
    return(1 - pnorm(qnorm(1 - alpha) - m))
  } else { # alt == "less"
    return(pnorm(qnorm(alpha) - m))
  }
}

SplSz.z.test <- function(delta, sigma, alpha, pwr, side = 2) {
  ceiling(((qnorm(1 - alpha/side) + qnorm(pwr)) * sigma / delta)^2)
}

SplSz.z.CI <- function(E, sigma, conf.level, side = 2) {
  ceiling((qnorm(1 - (1 - conf.level)/side) * sigma / E)^2)
}

# § 4-4.1 Hypothesis Testing on the Mean

# Figure 4-9 The p-value for a z-test. (a) The two-sided alternative H1: μ
# ≠ μ0. (b) The one-sided alternative H1: μ > μ0. (c) The one-sided
# alternative H1: μ < μ0.

# Figure 4-10 The distribution of Z0 when H0: μ = μ0 is true, with
# critical region for (a) the two-sided alternative H1: μ ≠ μ0. (b) The
# one-sided alternative H1: μ > μ0. (c) The one-sided alternative H1: μ
# < μ0.
```

```
# EXAMPLE 4-3 Propellant Burning Rate

# --- 3. Parameters ---
(params <- list(mu0 = 50, sigma = 2, alpha = 0.05, n = 25,
            delta = c(-1, 1), pwr.target = 0.90, E = 1.5,
            conf.target = 0.95)) # delta ≡ mu − mu0

delta_range <- seq(-3, 3, length.out = 100)
(n_test_sizes <- c(10, 25, 50, 100))

# --- 4. Basic Stats & Data Simulation ---
(xbar <- 51.3)
(se <- params$sigma / sqrt(params$n))
(z0 <- (xbar - params$mu0) / se)

set.seed(123)
Rt_raw <- rnorm(params$n, params$mu0, params$sigma)

# Rescale to have sample mean xbar
Rt <- xbar + (Rt_raw - mean(Rt_raw))

# --- 5. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n=== TWO-SIDED TEST ANALYSIS ===\n")
(CritVal_2s <- qnorm(c(params$alpha/2, 1 - params$alpha/2)))
(PVal_2s    <- 2 * (1 - pnorm(abs(z0))))
(CI_2s      <- xbar + CritVal_2s * se)
z.test(Rt, mu = params$mu0, sigma.x = params$sigma)
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_2s))

cat("Confidence Interval:", "[", CI_2s[1], ",", CI_2s[2], "]\n")

# 5.1 Power at δ = −1, 1:
Pwr.z.test(params$n, params$delta, params$sigma, params$alpha)
```

```
# 5.2. Power Curves for n = 10, 25, 50 and 100
power_2s_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = params$sigma,
                                                     alpha = params$alpha)))

# 5.3. Sample Size Requirements
# a. For Hypothesis Testing
(n_req_2s <- SplSz.z.test(params$delta, params$sigma, params$alpha,
              params$pwr.target, 2))
cat(sprintf("Required n for Power ≥ %.2f at δ = %s: %s\n",
      params$pwr.target, paste(params$delta, collapse = ", "),
      paste(n_req_2s, collapse = ", ")))

cat("\nPower at δ =", params$delta[1], "when n =", n_req_2s[1] - 1, ",",
  n_req_2s[1], ":",
  Pwr.z.test(c(n_req_2s[1] - 1, n_req_2s[1]), params$delta[1],
    params$sigma, params$alpha, "two.sided"),
  "\n", "\nPower at δ = ", params$delta[2], "when n =", n_req_2s[2] - 1,
  ",", n_req_2s[2], ":",
  Pwr.z.test(c(n_req_2s[2] - 1, n_req_2s[2]), params$delta[2],
    params$sigma, params$alpha, "two.sided"),
  "\n")

# b. For Confidence Interval
(SplSz_CI_2s <- SplSz.z.CI(params$E, params$sigma, params$conf.target, 2))
cat(sprintf("Required n for Error Margin = %s and Confidence ≥ %.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_2s))
```

```
cat("\nNote: {|X¯ − μ| ≤ E} = {μ ε [X¯ − E, X¯ + E]} and
      CI = [x¯ − E, x¯ + E] = [", xbar - params$E, ",", xbar + params$E,
"].
n =", SplSz_CI_2s - 1, ":", 100 * params$conf.target, "% CI = [",
  xbar + CritVal_2s[1] * params$sigma / sqrt(SplSz_CI_2s - 1), ",",
  xbar + CritVal_2s[2] * params$sigma / sqrt(SplSz_CI_2s - 1), "]
n =", SplSz_CI_2s, ":", 100 * params$conf.target, "% CI = [",
  xbar + CritVal_2s[1] * params$sigma / sqrt(SplSz_CI_2s), ",",
  xbar + CritVal_2s[2] * params$sigma / sqrt(SplSz_CI_2s), "]\n")

# 5.4. Visualizations
# a. Hypothesis Testing (Current n)
# Showing the Effect of δ on Power
normal.and.t.dist(params$mu0, params$mu0 + params$delta[1], xbar,
  params$sigma, params$n, NA, params$alpha/2, params$alpha/2, T, T, T)
normal.and.t.dist(params$mu0, params$mu0 + params$delta[2], xbar,
  params$sigma, params$n, NA, params$alpha/2,params$alpha/2, T, T, T)

# b. Confidence Interval (Current n)
normal.and.t.dist(NA, NA, xbar, params$sigma, params$n, NA, params$alpha/2,
  params$alpha/2, F, F, T, T, "c")

# c. Power Curves
ggplot(power_2s_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Z-Test (Known Variance)",
    subtitle =
    expression(H[0]*": "*mu*" = "*mu[0]*" vs. "*H[1]*": "*mu*" ≠ "*mu[0]),
    x = expression(delta == mu - mu[0]), y = "Power",
    color = "Sample Size (n)")
```

```
# d. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n at δ = −1, 1:
normal.and.t.dist(params$mu0, params$mu0 + params$delta[1], xbar,
  params$sigma, n_req_2s[1] - 1, NA, params$alpha/2, params$alpha/2, T, T,
  T)
normal.and.t.dist(params$mu0, params$mu0 + params$delta[1], xbar,
  params$sigma, n_req_2s[1], NA, params$alpha/2, params$alpha/2, T, T, T)
normal.and.t.dist(params$mu0, params$mu0 + params$delta[2], xbar,
  params$sigma, n_req_2s[1] - 1, NA, params$alpha/2, params$alpha/2, T, T,
  T)
normal.and.t.dist(params$mu0, params$mu0 + params$delta[2], xbar,
  params$sigma, n_req_2s[1], NA, params$alpha/2, params$alpha/2, T, T, T)

# e. Confidence Interval vs. Margin of Error
# Plotting n − 1 vs. required n
normal.and.t.dist(NA, NA, xbar, params$sigma, SplSz_CI_2s - 1, NA,
  params$alpha/2, params$alpha/2, F, F, T, T, "c")
normal.and.t.dist(NA, NA, xbar, params$sigma, SplSz_CI_2s, NA,
  params$alpha/2, params$alpha/2, F, T, T, T, "c")

# --- 6. One-Sided (Greater) Test Analysis ---
# ========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr <- qnorm(c(params$alpha, 1)))
(PVal_gr    <- 1 - pnorm(z0))
(CI_gr      <- xbar + CritVal_gr * se)
z.test(Rt, alternative = "greater", mu = params$mu0,
  sigma.x = params$sigma)
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_gr))

cat("Confidence Interval:", "[", CI_gr[1], ", ∞)\n")

# 6.1 Power at δ = 1:
Pwr.z.test(params$n, params$delta[2], params$sigma, params$alpha,
           "greater")
```

```
# 6.2. Power Curves for n = 10, 25, 50 and 100
power_gr_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = params$sigma,
                                                     alpha = params$alpha,
                                                     alt = "greater")))

# 6.3. Sample Size Requirements
# a. For Hypothesis Testing
(n_req_gr <- SplSz.z.test(params$delta[2], params$sigma, params$alpha,
              params$pwr.target, 1))
cat(sprintf("Required n for Power ≥ %.2f at δ = %s: %s\n",
      params$pwr.target, params$delta[2], n_req_gr))

cat("\nPower at δ =", params$delta[2], "when n =", n_req_gr - 1, ",",
  n_req_gr, ":",
  Pwr.z.test(c(n_req_gr - 1, n_req_gr), params$delta[2], params$sigma,
    params$alpha, "greater"),
  "\n")

# b. For Confidence Interval
(SplSz_CI_gr <- SplSz.z.CI(params$E, params$sigma, params$conf.target, 1))
cat(sprintf("Required n for Error Margin = %s and Confidence ≥ %.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_gr))

cat("\nNote: {X¯ − μ ≤ E} = {μ ε [X¯ − E, ∞)} and
      CI = [x¯ − E, ∞) = [", xbar - params$E, ",", "∞).
n =", SplSz_CI_gr - 1, ":", 100 * conf.target, "% CI = [",
  xbar + CritVal_gr[1] * params$sigma / sqrt(SplSz_CI_gr - 1), ", ∞)\
n =", SplSz_CI_gr, ":", 100 * params$conf.target, "% CI = [",
  xbar + CritVal_gr[1] * params$sigma / sqrt(SplSz_CI_gr), ", ∞)\n")

# 6.4. Visualizations
# a. Hypothesis Testing (Current n)
# Showing the Effect of δ on Power
normal.and.t.dist(params$mu0, params$mu0 + params$delta[2], xbar,
  params$sigma, params$n, NA, params$alpha, params$alpha, T, T)
```

```
# b. Confidence Interval (Current n)
normal.and.t.dist(NA, NA, xbar, params$sigma, params$n, NA, params$alpha,
  params$alpha, F, F, T, F, "c")

# c. Power Curves
ggplot(power_gr_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Z-Test (Known Variance)",
    subtitle =
     expression(H[0]*": "*mu*" ≤ "*mu[0]*" vs. "*H[1]*": "*mu*" > "*mu[0]),
    x = expression(delta == mu - mu[0]), y = "Power",
    color = "Sample Size (n)")

# d. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n at δ = 1:
normal.and.t.dist(params$mu0, params$mu0 + params$delta[2], xbar,
  params$sigma, n_req_gr - 1, NA, params$alpha, params$alpha, T, T)
normal.and.t.dist(params$mu0, params$mu0 + params$delta[2], xbar,
  params$sigma, n_req_gr, NA, params$alpha, params$alpha, T, T)

# e. Confidence Interval vs. Margin of Error
# Plotting n − 1 vs. required n
normal.and.t.dist(NA, NA, xbar, params$sigma, SplSz_CI_gr - 1, NA,
  params$alpha, params$alpha, F, F, T, F, "c")
normal.and.t.dist(NA, NA, xbar, params$sigma, SplSz_CI_gr, NA,
  params$alpha, params$alpha, F, T, T, F, "c")
```

```
# --- 7. One-Sided (Less) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le <- qnorm(c(0, 1 - params$alpha)))
(PVal_le    <- pnorm(z0))
(CI_le      <- xbar + CritVal_le * se)
z.test(Rt, alternative = "less", mu = params$mu0, sigma.x = params$sigma)
cat(sprintf("Test Statistic (z0): %.2f\nP-Value: %.4f\n", z0, PVal_le))

cat("Confidence Interval:", "(−∞,", CI_le[2], "]\n")

# 7.1 Power at δ = −1:
Pwr.z.test(params$n, params$delta[1], params$sigma, params$alpha, "less")

# 7.2. Power Curves for n = 10, 25, 50 and 100
power_le_df <- expand.grid(delta = delta_range, n = n_test_sizes) %>%
               mutate(power = mapply(Pwr.z.test, n = n, delta = delta,
                                     MoreArgs = list(sigma = params$sigma,
                                                     alpha = params$alpha,
                                                     alt = "less")))

# 7.3. Sample Size Requirements
# a. For Hypothesis Testing
(n_req_le <- SplSz.z.test(params$delta[1], params$sigma, params$alpha,
               params$pwr.target, 1))
cat(sprintf("Required n for Power ≥ %.2f at δ = %s: %s\n",
      params$pwr.target, params$delta[1], n_req_le))

cat("\nPower at δ =", params$delta[1], "when n =", n_req_le - 1, ",",
  n_req_le, ":",
  Pwr.z.test(c(n_req_le - 1, n_req_le), params$delta[1], params$sigma,
    params$alpha, "less"),
  "\n")
```

```
# b. For Confidence Interval
(SplSz_CI_le <- SplSz.z.CI(params$E, params$sigma, params$conf.target, 1))
cat(sprintf("Required n for Error Margin = %s and Confidence ≥ %.2f: %d\n",
      params$E, params$conf.target, SplSz_CI_le))

cat("\nNote: {X¯ − μ ≥ −E} = {μ ε (−∞, X¯ + E]} and
      CI = (−∞, x¯ + E] = (−∞,", xbar + params$E, "].
n =", SplSz_CI_le - 1, ":", 100 * params$conf.target, "% CI = (−∞,",
  xbar + CritVal_le[2] * params$sigma / sqrt(SplSz_CI_le - 1), "]
n =", SplSz_CI_le, ":", 100 * params$conf.target, "% CI = (−∞,",
  xbar + CritVal_le[2] * params$sigma / sqrt(SplSz_CI_gr), "]\n")

# 7.4. Visualizations
# a. Hypothesis Testing (Current n)
# Showing the Effect of δ on Power
normal.and.t.dist(params$mu0, params$mu0 + delta_val[1], xbar,
  params$sigma, params$n, NA, params$alpha, params$alpha, T, T, T, F)

# b. Confidence Interval (Current n)
normal.and.t.dist(NA, NA, xbar, params$sigma, params$n, NA, params$alpha,
  params$alpha, F, T, F, T, "c")

# c. Power Curves
ggplot(power_le_df, aes(x = delta, y = power, color = as.factor(n))) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = params$pwr.target, linetype = "dashed",
    color = "red") +
  labs(title = "Power Curves for Z-Test (Known Variance)",
    subtitle =
     expression(H[0]*": "*mu*" ≥ "*mu[0]*" vs. "*H[1]*": "*mu*" < "*mu[0]),
    x = expression(delta == mu - mu[0]), y = "Power",
    color = "Sample Size (n)")
```

```
# d. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n at δ = −1:
normal.and.t.dist(params$mu0, params$mu0 + delta_val[1], xbar,
  params$sigma, n_req_le - 1, NA, params$alpha, params$alpha, T, T, T, F)
normal.and.t.dist(params$mu0, params$mu0 + delta_val[1], xbar,
  params$sigma, n_req_le, NA, params$alpha, params$alpha,T, T, T, F)

# e. Confidence Interval vs. Margin of Error
# Plotting n − 1 vs. required n
normal.and.t.dist(NA, NA, xbar, params$sigma, SplSz_CI_le - 1, NA,
  params$alpha, params$alpha, F, F, F, T, "c")
normal.and.t.dist(NA, NA, xbar, params$sigma, SplSz_CI_le, NA,
  params$alpha, params$alpha, F, F, F, T, "c")

# § 4-4.2 Type II Error and Choice of Sample Size

# Figure 4-11 The distribution of Z0 under H0 and H1.

# EXAMPLE 4-4 Sample Size for the Propellant Burning Rate Problem

# Refer to EXAMPLE 4-3.

# Table 4-2 Minitab Computations

# § 4-4.3 Large-Sample Test

# § 4-4.4 Some Practical Comments on Hypothesis Testing

# § 4-4.5 Confidence Interval on the Mean

# Figure 4-12 Repeated construction of a confidence interval for μ.

# Displays 100 simulated two-sided 100(1 − α)% confidence intervals.
CIplot(100, params$n, params$mu0 + params$delta[1], params$sigma)

CIplot(100, params$n, params$mu0 + params$delta[2], params$sigma)
```

```
# Displays 100 simulated two-sided 100(1 − 2α)% confidence intervals,
# where the lower and upper confidence limits are the same as the one-sided
# 100(1 − α)% lower and upper confidence bounds, respectively.
CIplot(100, params$n, params$mu0 + params$delta[1], params$sigma,
  1 - 2 * params$alpha)

CIplot(100, params$n, params$mu0 + params$delta[2], params$sigma,
  1 - 2 * params$alpha)

# Figure 4-13 The distribution of Z.

# EXAMPLE 4-5 Propellant Burning Rate

# Refer to EXAMPLE 4-3.

# Figure 4-14 Error in estimating μ with xbar.

# Sample Size for a Specified E on the Mean, Variance Known

# EXAMPLE 4-6 Propellant Burning Rate

# Refer to EXAMPLE 4-3.

# § 4-4.6 General Method for Deriving a Confidence Interval

# EXERCISES FOR § 4-4
```

```
# § 4-5 Inference on the Mean of a Population, Variance Unknown

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(car, HH)

# --- 2. Function ---
Pwr.t.test <- function (n, delta, sigma, alpha, alt = "two.sided") {
  df  <- n - 1
  ncp <- delta / (sigma / sqrt(n))
  if (alt == "two.sided") {
    return(1 - pt(qt(1 - alpha/2, df), df, ncp) +
           pt(qt(alpha/2, df), df, ncp))
  } else if (alt == "greater") {
    return(1 - pt(qt(1 - alpha, df), df, ncp))
  } else { # alt == "less"
    return(pt(qt(alpha, df), df, ncp))
  }
}

# § 4-5.1 Hypothesis Testing on the Mean

# Figure 4-15 Probability density functions of several t distributions.

# --- 3. Plotting t Distributions ---
old.par <- par(oma = c(0, 0.5, 0, 0), mar = c(5, 4, 4, 6) + .1)
norm.setup(main =
  expression("standard t density, df = 1, 10 and" ~ infinity), df.t = 1)
norm.curve(df.t = 1, col = 1)
norm.curve(df.t = 10, col = 10)
norm.curve(col = 11)

norm.setup(main =
  expression("standard t density, df = 1, 2, ⋯, 10 and" ~ infinity),
  df.t = 1)
for (i in 1:10) norm.curve(df.t = i, col = i)
norm.curve(col = 11)
```

```
norm.setup(df.t = 1, main =
expression("standard t density, df = 1, 10, 11, 12, …, 20 and" ~
infinity))
norm.curve(col = 1, df.t = 1)
norm.curve(col = 10, df.t = 10)
for (i in 11:20) norm.curve(col = i - 10, df.t = i)
norm.curve(col = 11)

norm.setup(df.t = 1, main =
expression("standard t density, df = 1, 10, 21, 22, …, 30 and" ~
infinity))
norm.curve(col = 1, df.t = 1)
norm.curve(col = 10, df.t = 10)
for (i in 21:30) norm.curve(col = i - 20, df.t = i)
norm.curve(col = 11)

norm.setup(df.t = 1, main =
expression("standard t density, df = 1, 10, 40, 60, 120 and" ~ infinity))
norm.curve(col = 1, df.t = 1)
norm.curve(col = 10, df.t = 10)
for (i in c(40, 60, 120)) norm.curve(col = i/20, df.t = i)
norm.curve(col = 11)

par(old.par)

# Figure 4-16 Percentage points of the t distribution.

# Figure 4-17 Calculating the P-value for a t-test: (a) H1: μ ≠ μ0;
# (b) H1: μ > μ0; (c) H1: μ < μ0.

# Figure 4-18 P-value for t0 = 2.719 and an upper-tailed test is shown to
# be between 0.005 and 0.01.

# Figure 4-19 The distribution of T_0 when H0: μ = μ0 is true, with
# critical region for (a) H_1: μ ≠ μ0, (b) H1: μ > μ0, and (c) H1: μ
# < μ0.
```

```
# EXAMPLE 4-7 Golf Clubs

# Figure 4-20 Normal probability plot of the coefficient of restitution
# data from Example 4-7.

# --- 4. Data & Parameters ---
(COR <- c(0.8411, 0.8580, 0.8042, 0.8191, 0.8532, 0.8730, 0.8182, 0.8483,
         0.8282, 0.8125, 0.8276, 0.8359, 0.8750, 0.7983, 0.8660))
(params <- list(mu0 = 0.82, mu = 0.83, sigma = 0.025, alpha = 0.05,
            n = length(COR), df = length(COR) - 1, delta = c(-0.02, 0.02),
            pwr.target = 0.8))

# --- 5. Basic Stats & Data Simulation ---
summary(COR)
(xbar <- mean(COR))
(s <- sd(COR))
(se <- s / sqrt(params$n))
(t0 <- (xbar - params$mu0) / se)

# Refer to § 4-6.2 for the 95% one-sided upper confidence bound on σ.
(CI.sigma <- c(0, sqrt(params$df * s^2 / qchisq(params$alpha, params$df))))

set.seed(123)
x10_raw <- rnorm(10, params$mu, params$sigma)
x11_raw <- rnorm(11, params$mu, params$sigma)
x13_raw <- rnorm(13, params$mu, params$sigma)
x14_raw <- rnorm(14, params$mu, params$sigma)

# Rescale to have sample mean xbar and sample standard deviation s
x10 <- xbar + s * (x10_raw - mean(x10_raw)) / sd(x10_raw)
x11 <- xbar + s * (x11_raw - mean(x11_raw)) / sd(x11_raw)
x13 <- xbar + s * (x13_raw - mean(x13_raw)) / sd(x13_raw)
x14 <- xbar + s * (x14_raw - mean(x14_raw)) / sd(x14_raw)

# 5.3 Normality Test
shapiro.test(COR)
qqPlot(COR)
```

```
# --- 6. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n=== TWO-SIDED TEST ANALYSIS ===\n")
(CritVal_2s <- qt(c(alpha/2, 1 - alpha/2), df))
(PVal_2s    <- 2 * (1 - pt(abs(t0), df)))
(CI_2s      <- xbar + CritVal_2s * se)
t.test(COR, alternative = "two.sided", mu = mu0)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_2s))

cat("Confidence Interval:", "[", CI_2s[1], ",", CI_2s[2], "]\n")

# 6.1 Power checks at δ = −1, 1:
Pwr.t.test(params$n, params$delta[1], c(s, CI.sigma[2]), params$alpha,
  "two.sided")
power.t.test(n = params$n, delta = params$delta[1], sd = c(s, CI.sigma[2]),
  sig.level = params$alpha, type = "one.sample", alternative = "two.sided")
Pwr.t.test(params$n, params$delta[2], c(s, CI.sigma[2]), params$alpha,
  "two.sided")
power.t.test(n = params$n, delta = params$delta[2], sd = c(s, CI.sigma[2]),
  sig.level = params$alpha, type = "one.sample", alternative = "two.sided")

# 6.2. Sample Size Requirements
cat(sprintf("Required n for Power ≥ %.2f at δ = %s:\n", params$pwr.target,
            paste(params$delta, collapse = ", ")))

power.t.test(delta = params$delta[1], sd = s, sig.level = params$alpha,
  power = params$pwr.target, type = "one.sample",
  alternative = "two.sided")
power.t.test(delta = params$delta[2], sd = s, sig.level = params$alpha,
  power = params$pwr.target, type = "one.sample",
  alternative = "two.sided")
```

```
Pwr.t.test(c(13, 14), params$delta[1], s, params$alpha, "two.sided")
power.t.test(n = c(13, 14), delta = params$delta[1], sd = s,
  sig.level = params$alpha, type = "one.sample", alternative = "two.sided")
Pwr.t.test(c(13, 14), params$delta[2], s, params$alpha, "two.sided")
power.t.test(n = c(13, 14), delta = params$delta[2], sd = s,
  sig.level = params$alpha, type = "one.sample", alternative = "two.sided")

power.t.test(delta = params$delta[1], sd = CI.sd[2],
  sig.level = params$alpha, power = params$pwr.target, type = "one.sample",
  alternative = "two.sided")
power.t.test(delta = params$delta[2], sd = CI.sd[2],
  sig.level = params$alpha, power = params$pwr.target, type = "one.sample",
  alternative = "two.sided")

Pwr.t.test(c(27, 28), params$delta[1], CI.sd[2], params$alpha, "two.sided")
power.t.test(n = c(27, 28), delta = params$delta[1], sd = CI.sd[2],
  sig.level = params$alpha, type = "one.sample", alternative = "two.sided")
Pwr.t.test(c(27, 28), params$delta[2], CI.sd[2], params$alpha, "two.sided")
power.t.test(n = c(27, 28), delta = params$delta[2], sd = CI.sd[2],
  sig.level = params$alpha, type = "one.sample", alternative = "two.sided")

# 6.3. Visualizations
# 6.3.1. Hypothesis Testing (Current n)
# Showing the Effect of δ on Power
NTplot(t.test(COR, mu = mu0), mean1 = mu1[1], zaxis = T, power = T)
NTplot(t.test(COR, mu = mu0), mean1 = mu1[2], zaxis = T, power = T)

# 6.3.2. Confidence Interval (Current n)
NTplot(t.test(COR, mu = mu0), type = "confidence", zaxis = T)

# 6.3.3. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n at δ = −0.02, 0.02:
NTplot(t.test(x13, mu = mu0), mean1 = params$mu0 + params$delta[1],
  zaxis = T, power = T)
NTplot(t.test(x14, mu = mu0), mean1 = params$mu0 + params$delta[1],
  zaxis = T, power = T)
```

```
NTplot(t.test(x13, mu = mu0), mean1 = params$mu0 + params$delta[2],
  zaxis = T, power = T)
NTplot(t.test(x14, mu = mu0), mean1 = params$mu0 + params$delta[2],
  zaxis = T, power = T)

# --- 7. One-Sided (Greater) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr <- qt(1 - alpha, df))
(PVal_gr    <- 1 - pt(t0, df))
(CI_gr      <- c(xbar - CritVal_gr * se, Inf))
t.test(COR, alternative = "greater", mu = mu0)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_gr))

cat("Confidence Interval:", "[", CI_gr[1], ", ∞)\n")

# 7.1 Power checks at δ = 1:
Pwr.t.test(params$n, params$delta[2], c(s, CI.sd[2]), params$alpha,
  "greater")
power.t.test(n = params$n, delta = params$delta[2], sd = c(s, CI.sd[2]),
  sig.level = params$alpha, type = "one.sample", alternative = "one.sided")

# 7.2. Sample Size Requirements
cat(sprintf("Required n for Power ≥ %.2f at δ = %s:\n", params$pwr.target,
            params$delta[2]))

power.t.test(delta = params$delta[2], sd = s, sig.level = params$alpha,
  power = params$pwr.target, type = "one.sample",
  alternative = "one.sided")

Pwr.t.test(c(10, 11), params$delta[2], s, params$alpha, "greater")
power.t.test(n = c(10, 11), delta = params$delta[2], sd = s,
  sig.level = params$alpha, type = "one.sample", alternative = "one.sided")
```

```
power.t.test(delta = params$delta[2], sd = CI.sd[2],
  sig.level = params$alpha, power = params$pwr.target, type = "one.sample",
  alternative = "one.sided")

Pwr.t.test(c(21, 22), params$delta[2], CI.sd[2], params$alpha, "greater")
power.t.test(n = c(21, 22), delta = params$delta[2], sd = CI.sd[2],
  sig.level = params$alpha, type = "one.sample", alternative = "one.sided")

# 7.3. Visualizations
# 7.3.1. Hypothesis Testing (Current n)
# Showing the Effect of δ on Power
NTplot(t.test(COR, alternative = "greater", mu = mu0), mean1 = mu1[2],
  zaxis = T, power = T)

# 7.3.2. Confidence Interval (Current n)
NTplot(t.test(COR, alternative = "greater", mu = mu0), type = "confidence",
   zaxis = T)

# 7.3.3. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n at δ = 0.02:
NTplot(t.test(x10, alternative = "greater", mu = mu0),
  mean1 = params$mu0 + params$delta[2], zaxis = T, power = T)
NTplot(t.test(x11, alternative = "greater", mu = mu0),
  mean1 = params$mu0 + params$delta[2], zaxis = T, power = T)

# --- 8. One-Sided (Less) Test Analysis ---
# ========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le <- qt(alpha, df))
(PVal_le    <- pt(t0, df))
(CI_le      <- c(-Inf, xbar - CritVal_le * se))
t.test(COR, alternative = "less", mu = mu0)
cat(sprintf("Test Statistic (t0): %.3f\nP-Value: %.4f\n", t0, PVal_le))

cat("Confidence Interval:", "(−∞,", CI_le[2], "]\n")
```

```
# 8.1 Power checks at δ = −1:
Pwr.t.test(params$n, params$delta[1], c(s, CI.sd[2]), params$alpha, "less")
power.t.test(n = params$n, delta = abs(params$delta[1]),
  sd = c(s, CI.sd[2]), sig.level = params$alpha, type = "one.sample",
  alternative = "one.sided")

# 8.2. Sample Size Requirements
cat(sprintf("Required n for Power ≥ %.2f at δ = %s:\n", params$pwr.target,
            params$delta[1]))

power.t.test(delta = abs(params$delta[1]), sd = s,
  sig.level = params$alpha, power = params$pwr.target, type = "one.sample",
  alternative = "one.sided")
Pwr.t.test(c(10, 11), params$delta[1], s, params$alpha, "less")
power.t.test(n = c(10, 11), delta = abs(params$delta[1]), sd = s,
  sig.level = params$alpha, type = "one.sample", alternative = "one.sided")

power.t.test(delta = abs(params$delta[1]), sd = CI.sd[2],
  sig.level = params$alpha, power = params$pwr.target, type = "one.sample",
  alternative = "one.sided")
Pwr.t.test(c(21, 22), params$delta[1], CI.sd[2], params$alpha, "less")
power.t.test(n = c(21, 22), delta = abs(params$delta[1]), sd = CI.sd[2],
  sig.level = params$alpha, type = "one.sample", alternative = "one.sided")

# 8.3. Visualizations
# 8.3.1. Hypothesis Testing (Current n)
# Showing the Effect of δ on Power
NTplot(t.test(COR, alternative = "less", mu = mu0), mean1 = mu1[1],
  zaxis = T, power = T)

# 8.3.2. Confidence Interval (Current n)
NTplot(t.test(COR, alternative = "less", mu = mu0), type = "confidence",
  zaxis = T)
```

```
# 8.3.3. Comparison of Sample Sizes (Targeting Required n)
# Plotting n − 1 vs. required n at δ = −0.02:
NTplot(t.test(x10, alternative = "less", mu = mu0),
  mean1 = params$mu0 + params$delta[1], zaxis = T, power = T)
NTplot(t.test(x11, alternative = "less", mu = mu0),
  mean1 = params$mu0 + params$delta[1], zaxis = T, power = T)

# 4-5.2 Type II Error and Choice of Sample Size

# EXAMPLE 4-8 Golf Clubs

# Refer to EXAMPLE 4-7.

# § 4-5.3 Confidence Interval on the Mean

# Displays 100 simulated two-sided 100(1 − α)% confidence intervals.
CIplot(100, params$n, params$mu, params$sigma)

# Displays 100 simulated two-sided 100(1 − 2α)% confidence intervals,
# where the lower and upper confidence limits are the same as the one-sided
# 100(1 − α)% lower and upper confidence bounds, respectively.
CIplot(100, params$n, params$mu, params$sigma, 1 - 2 * params$alpha)

# EXAMPLE 4-9 Golf Clubs

# Refer to EXAMPLE 4-7.

# EXERCISES FOR § 4-5
```

```
# § 4-6 Inference on the Variance of a Normal Population

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(HH)

# --- 2. Function ---
Pwr.chisq.test <- function (n, r, alpha, alt = "two.sided") {
  df  <- n - 1
  if (alt == "two.sided") {
    return(1 - pchisq(qchisq(1 - alpha/2, df) / r^2, df) +
           pchisq(qchisq(alpha/2, df) / r^2, df))
  } else if (alt == "greater") {
    return(1 - pchisq(qchisq(1 - alpha, df) / r^2, df))
  } else { # alt == "less"
    return(pchisq(qchisq(alpha, df) / r^2, df))
  }
}

# § 4-6.1 Hypothesis Testing on the Variance of a Normal Population

# Figure 4-21 Probability density functions of several χ² distributions.

# --- 3. Plotting χ² Distributions ---
old.omd <- par(omd = c(0.03, .88, 0, 1))

chisq.setup(xlim.in = c(0, qchisq(1 - .01, 10)),
  main.in = expression("Chisq density:" ~~ nu=="2, 5 and 10"))
chisq.curve(2, col = "blue")
chisq.curve(5, col = 2)
chisq.curve(10, col = 3)
```

```
chisq.setup(xlim.in = c(0, qchisq(1 - .01, 10)),
  main.in = expression("Chisq density:" ~~ nu=="1, 2, ⋯, 10"))
for (i in 1:10) chisq.curve(i, col = i)
chisq.curve(2, col = "blue")
chisq.curve(5, col = 2)
chisq.curve(10, col = 3)

chisq.setup(xlim.in = c(0, qchisq(1 - .01, 20)),
  main.in = expression("Chisq density:" ~~ nu=="2, 5, 10, 11, 12, ⋯, 20"))
for (i in 1:10) chisq.curve(i + 10, col = i)
chisq.curve(2, col = "blue")
chisq.curve(5, col = 2)
chisq.curve(10, col = 3)

chisq.setup(xlim.in = c(0, qchisq(1 - .01, 30)),
  main.in = expression("Chisq density:" ~~ nu=="2, 5, 10, 21, 22, ⋯, 30"))
for (i in 1:10) chisq.curve(i + 20, col = i)
chisq.curve(2, col = "blue")
chisq.curve(5, col = 2)
chisq.curve(10, col = 3)

chisq.setup(xlim.in = c(0, qchisq(1 - .01, 100)),
 main.in = expression("Chisq density:" ~~ nu=="2, 5, 10, 20, 30, ⋯, 100"))
for (i in 1:10) chisq.curve(i*10, col = i)
chisq.curve(2, col = "blue")
chisq.curve(5, col = 2)
chisq.curve(10, col = 3)

par(old.omd)

# Figure 4-22 Percentage point χ²_{α, k} of the χ² distribution.

# Figure 4-23 Distribution of the test statistic for H_0: σ² = σ²_0 with
# critical region values for (a) H_1: σ² ≠ σ²_0, (b) H_1: σ² > σ²_0,
# and (c) H_1: σ² < σ²_0.

# EXAMPLE 4-10 Bottle Filling
```

```
# --- 4. Parameters ---
(params <- list(mu0 = 1, sigma0 = sqrt(0.01), alpha = 0.05, n = 20,
            df = 20 - 1, r = c(2/3, 3/2), pwr.target = 0.8))

# --- 5. Basic Stats & Data Simulation ---
(s <- sqrt(0.0153))
(chisq0 <- params$df * s^2 / params$sigma0^2)
(qch <- range(c(chisq0, qchisq(1 - pchisq(chisq0, params$df), params$df))))

set.seed(123)
Vol_raw <- rnorm(params$n, params$mu0, params$sigma0)

# Rescale to have sample mean mean(Vol_raw) and sample standard deviation s
Vol <- mean(Vol_raw) + s * (Vol_raw - mean(Vol_raw)) / sd(Vol_raw)

# --- 6. Two-Sided Test Analysis ---
# ==========================================================================
# CASE: TWO-SIDED TEST
# ==========================================================================
cat("\n=== TWO-SIDED TEST ANALYSIS ===\n")
(CritVal_2s   <- qchisq(c(params$alpha/2, 1 - params$alpha/2), params$df))
(PVal_2s      <- 2 * min(pchisq(chisq0, df),
                         1 - pchisq(chisq0, params$df)))
(CI.sigma2_2s <- params$df * s^2 /
                qchisq(c(1 - params$alpha/2, params$alpha/2), params$df))
(CI.sigma_2s  <- sqrt(CI.sigma2_2s))
cat(sprintf("Test Statistic (χ²0): %.2f\nP-Value: %.4f\n", chisq0,
      PVal_2s))

cat("Confidence Interval for σ²:", "[", CI.sigma2_2s[1], ",",
    CI.sigma2_2s[2], "]\n")

cat("Confidence Interval for σ:", "[", CI.sigma_2s[1], ",",
    CI.sigma_2s[2], "]\n")

# 6.1 Power at r (≡ σ₁/σ₀) = 2/3, 3/2:
Pwr.chisq.test(params$n, params$r, params$alpha, "two.sided")
```

```
# 6.2. Visualizations
old.omd <- par(omd = c(.03, .88, 0.1, 1))
chisq.setup(params$df)
chisq.curve(params$df, 0, F, c(params$alpha/2, params$alpha/2),
  col = "blue")
chisq.observed(qch[1], "green", params$df, 0, F, "chisq", "left")
chisq.observed(qch[2], 2, params$df, 0, F, "chisq", "right", .0649)
axis(3, 18, expression(n==nu + 1 ~~ sigma[0]^2==.01 ~~ s^2==.0153 ~~
chi[0]^2==nu ~ s^2 / sigma[0]^2), T, 1.5, lwd = 0)
axis(1, qchisq(c(params$alpha/2, 1 - params$alpha/2), params$df),
  round(params$df * s^2 / qchisq(c(params$alpha/2, 1 - params$alpha/2),
    params$df), 4),
  T, 5, lwd = 0)
axis(1, -1.4, expression(nu ~ s^2/chi[nu]^2), T, 5, lwd = 0)
axis(1, qchisq(c(params$alpha/2, 1 - params$alpha/2), params$df),
  round(sqrt(params$df * s^2 /
             qchisq(c(params$alpha/2, 1 - params$alpha/2), params$df)), 4),
  T, 6.5, lwd = 0)
axis(1, -1.4, expression(sqrt(nu) ~ s/sqrt(chi[nu]^2)), T, 6.5, lwd = 0)
axis(3, 18,
  expression(n==nu + 1 ~~ s^2==.0153 ~~ LCL ~~ UCL==nu ~ s^2/chi[nu]^2), T,
  -2.5, lwd = 0)
par(old.omd)

# --- 7. One-Sided (Greater) Test Analysis ---
# ========================================================================
# CASE: ONE-SIDED (GREATER) TEST
# ========================================================================
cat("\n=== ONE-SIDED (GREATER) TEST ANALYSIS ===\n")
(CritVal_gr   <- qchisq(1 - params$alpha, params$df))
(PVal_gr      <- 1 - pchisq(chisq0, params$df))
(CI.sigma2_gr <- c(params$df * s^2 / qchisq(1 - params$alpha, params$df),
                   Inf))
(CI.sigma_gr  <- sqrt(CI.sigma2_gr))
cat(sprintf("Test Statistic (χ²0): %.2f\nP-Value: %.4f\n", chisq0,
      PVal_gr))
```

```
cat("Confidence Interval for σ²:", "[", CI.sigma2_gr[1], ", ∞)\n")

cat("Confidence Interval for σ:", "[", CI.sigma_gr[1], ", ∞)\n")

# 7.1 Power at r (≡ σ₁/σ₀) = 3/2:
Pwr.chisq.test(params$n, params$r[2], params$alpha, "greater")

# 7.2. Visualizations
old.omd <- par(omd = c(.03, .88, .1, 1))
chisq.setup(params$df)
chisq.curve(params$df, col = "blue")
chisq.observed(chisq0, "green", params$df)
axis(3, 18, expression(n==nu + 1 ~~ sigma[0]^2==.01 ~~ s^2==.0153 ~~
chi[0]^2==nu ~ s^2/sigma[0]^2), T, 1.5, lwd = 0)
axis(1, qchisq(1 - params$alpha, params$df),
  round(params$df * s^2 / qchisq(1 - params$alpha, params$df), 4), T, 5,
  lwd = 0)
axis(1, -1.4, expression(nu ~ s^2/chi[nu]^2), T, 5, lwd = 0)
axis(1, qchisq(1 - params$alpha, params$df),
  round(sqrt(params$df * s^2 / qchisq(1 - params$alpha, params$df)), 4), T,
  6.5, lwd = 0)
axis(1, -1.4, expression(sqrt(nu) ~ s/sqrt(chi[nu]^2)), T, 6.5, lwd = 0)
axis(3, 18, expression(n==nu + 1 ~~ s^2==.0153 ~~ LCL==nu ~ s^2/chi[nu]^2),
  T, -2.5, lwd = 0)
par(old.omd)

# --- 8. One-Sided (Less) Test Analysis ---
# ==========================================================================
# CASE: ONE-SIDED (LESS) TEST
# ==========================================================================
cat("\n=== ONE-SIDED (LESS) TEST ANALYSIS ===\n")
(CritVal_le   <- qchisq(params$alpha, params$df))
(PVal_le      <- pchisq(chisq0, params$df))
(CI.sigma2_le <- c(0, params$df * s^2 / qchisq(params$alpha, params$df)))
(CI.sigma_le  <- sqrt(CI.sigma2_le))
cat(sprintf("Test Statistic (χ²0): %.2f\nP-Value: %.4f\n", chisq0,
      PVal_le))
```

```
cat("Confidence Interval for σ²:", "(0,", CI.sigma2_le[2], "]\n")

cat("Confidence Interval for σ:", "(0,", CI.sigma_le[2], "]\n")

# 8.1 Power at r (≡ σ₁/σ₀) = 2/3:
Pwr.chisq.test(params$n, params$r[1], params$alpha, "less")

# 8.2. Visualizations
old.omd <- par(omd = c(.03, .88, .1, 1))
chisq.setup(params$df)
chisq.curve(params$df, 0, F, 1 - params$alpha, shade = "left",
  col = "blue")
chisq.observed(chisq0, "green", params$df, 0, F, "chisq", "left")
axis(3, 18, expression(n==nu + 1 ~~ sigma[0]^2==.01 ~~ s^2==.0153 ~~
chi[0]^2==nu ~ s^2/sigma[0]^2), T, 1.5, lwd = 0)
axis(1, qchisq(params$alpha, params$df),
  round(params$df * s^2 / qchisq(params$alpha, params$df), 4), T, 5,
  lwd = 0)
axis(1, -1.4, expression(nu ~ s^2/chi[nu]^2), T, 5, lwd = 0)
axis(1, qchisq(params$alpha, params$df),
  round(sqrt(params$df * s^2 / qchisq(params$alpha, params$df)), 4), T,
  6.5, lwd = 0)
axis(1, -1.4, expression(sqrt(nu) ~ s/sqrt(chi[nu]^2)), T, 6.5, lwd = 0)
axis(3, 18, expression(n==nu + 1 ~~ s^2==.0153 ~~ UCL==nu ~ s^2/chi[nu]^2),
  T, -2.5, lwd = 0)
par(old.omd)

# § 4-6.2 Confidence Interval on the Variance of a Normal Population

# EXAMPLE 4-11 Bottle Filling

# Refer to EXAMPLE 4-10.

# EXERCISES FOR § 4-6
```

```
# § 4-7 Inference on a Population Proportion

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(car, HH, Hmisc)

# § 4-7.1 Hypothesis Testing on a Binomial Proportion

# Aprx Pwr for Two- or One-Sided Alt Hypothesis on a Pop Proportion
# Using an Aprx Z-Test

AprxPwrAprx.z.test <- function (n, p0, p1, aprx.lv, alt) {
se0 <- sqrt(p0*(1 - p0)/n); se1 <- sqrt(p1*(1 - p1)/n)
m <- (p1 - p0)/se0; sd <- se1/se0
if (alt == 2) 1 - pnorm(qnorm(1 - aprx.lv/2), m, sd) +
pnorm(qnorm(aprx.lv/2), m, sd)
else 1 - pnorm(qnorm(1 - aprx.lv), abs(m), sd)}

# EXAMPLE 4-12 Engine Controllers

(p0 <- .05); (aprx.lv <- .05); (n <- 200); (x <- 4)

(p1 <- c(.07, .03)); (delta <- p1 - p0)

(pwr.target <- .9)

# Normal approx.:

(phat <- x/n); (sd0 <- sqrt(p0*(1 - p0))); (se0 <- sd0/sqrt(n))

(z0 <- (phat - p0)/se0)

(sd1 <- sqrt(p1*(1 - p1))); (se1 <- sd1/sqrt(n))

(sdhat <- sqrt(phat*(1 - phat))); (sehat <- sdhat/sqrt(n))

(a <- c(aprx.lv, aprx.lv/2))
```

```
(ptilde <- (phat + qnorm(a)^2/(2*n))/(1 + qnorm(a)^2/n))

(setilde <- sqrt(sehat^2 + qnorm(a)^2/(4*n^2))/(1 + qnorm(a)^2/n))

(sdtilde <- sqrt(n)*setilde)

# alt = "two.sided":

prop.test(x, n, p0, "t", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(c(aprx.lv/2, 1 - aprx.lv/2))) # Aprx

# Reject H0 if and ony if "z0 < CritVal[1] or z0 > CritVal[2]".

(CritVal.x0 <- n*p0 + CritVal*sqrt(n)*sd0) # Aprx

(CritVal.x <- c(ceiling(CritVal.x0[1]), floor(CritVal.x0[2]))) # Aprx

# Reject H0 if and only if "x < CritVal.x[1] or x > CritVal.x[2]".

(Alpha <-
1 - (pbinom(CritVal.x[2], n, p0) - pbinom(CritVal.x[1] - 1, n, p0)))

(AprxPVal <- 2*(1 - pnorm(abs(z0))))

(PVal <- 2*min(1/2, pbinom(x, n, p0), 1 - pbinom(x - 1, n, p0)))

AprxPwrAprx.z.test(n, p0, p1, aprx.lv, 2)

(Pwr <-
1 - (pbinom(CritVal.x[2], n, p1) - pbinom(CritVal.x[1] - 1, n, p1)))

(AsymptoticCI <- phat + CritVal*sehat) # Aprx

(AgrestiCoullCI <- ptilde[2] + CritVal*setilde[2]) # Aprx

binconf(x, n, alpha, "all", T, T) # Aprx
```

```
NTplot(p0 = p0, p1 = p1[1], p.hat = phat, n = n,
xlim = range(phat, p0 + c(-3, 3)*se0, p1[1] + c(-3, 3)*se1[1]),
ylim = range(0, dnorm(p0, p0, se0), dnorm(p1[1], p1[1], se1[1])),
alpha.right = aprx.lv/2, alpha.left = aprx.lv/2, distribution.name = "b",
zaxis = T, zlaxis = T, power = T) # Aprx

NTplot(p0 = p0, p1 = p1[2], p.hat = phat, n = n,
xlim = range(phat, p0 + c(-3, 3)*se0, p1[2] + c(-3, 3)*se1[2]),
ylim = range(0, dnorm(p0, p0, se0), dnorm(p1[2], p1[2], se1[2])),
alpha.right = aprx.lv/2, alpha.left = aprx.lv/2, distribution.name = "b",
zaxis = T, zlaxis = T, power = T) # Aprx

NTplot(p.hat = phat, n = n, xlim = phat + c(-3, 3)*sehat,
ylim = c(0, dnorm(phat, phat, sehat)), distribution.name = "b", type = "c",
zaxis = T) # Aprx

# alt = "gr":

prop.test(x, n, p0, "g", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(1 - aprx.lv)) # Aprx

# Reject H0 if and ony if "z0 > CritVal".

(CritVal.x0 <- n*p0 + CritVal*sqrt(n)*sd0) # Aprx

(CritVal.x <- floor(CritVal.x0)) # Aprx

# Reject H0 if and only if "x > CritVal.x".

(Alpha <- 1 - pbinom(CritVal.x, n, p0))

(AprxPVal <- 1 - pnorm(z0))

(PVal <- 1 - pbinom(x - 1, n, p0))

AprxPwrAprx.z.test(n, p0, p1[1], aprx.lv, 1)
```

```
(Pwr <- 1 - pbinom(CritVal.x, n, p1[1]))

(AsymptoticCI <- c(phat - CritVal*sehat, 1)) # Aprx

(AgrestiCoullCI <- c(ptilde[1] - CritVal*setilde[1], 1)) # Aprx

binconf(x, n, 2*aprx.lv, "all", T, T) # Aprx

NTplot(p0 = p0, p1 = p1[1], p.hat = phat, n = n,
xlim = range(phat, p0 + c(-3, 3)*se0, p1[1] + c(-3, 3)*se1[1]),
ylim = range(0, dnorm(p0, p0, se0), dnorm(p1[1], p1[1], se1[1])),
distribution.name = "b", zaxis = T, zlaxis = T, power = T) # Aprx

NTplot(p.hat = phat, n = n, xlim = phat + c(-3, 3)*sehat,
ylim = c(0, dnorm(phat, phat, sehat)), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", type = "c", zaxis = T)
# Aprx

# alt = "less":

prop.test(x, n, p0, "l", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(aprx.lv)) # Aprx

# Reject H0 if and ony if "z0 < CritVal".

(CritVal.x0 <- n*p0 + CritVal*sqrt(n)*sd0) # Aprx

(CritVal.x <- ceiling(CritVal.x0)) # Aprx

# Reject H0 if and only if "x < CritVal.x".

(Alpha <- pbinom(CritVal.x - 1, n, p0))

(AprxPVal <- pnorm(z0))

(PVal <- pbinom(x, n, p0))
```

```
AprxPwrAprx.z.test(n, p0, p1[2], aprx.lv, 1)

(Pwr <- pbinom(CritVal.x - 1, n, p1[2]))

(AsymptoticCI <- c(0, phat - CritVal*sehat)) # Aprx

(AgrestiCoullCI <- c(0, ptilde[1] - CritVal*setilde[1])) # Aprx

binconf(x, n, 2*aprx.lv, "all", T, T) # Aprx

NTplot(p0 = p0, p1 = p1[2], p.hat = phat, n = n,
xlim = range(phat, p0 + c(-3, 3)*se0, p1[2] + c(-3, 3)*se1[2]),
ylim = range(0, dnorm(p0, p0, se0), dnorm(p1[2], p1[2], se1[2])),
alpha.right = 0, alpha.left = aprx.lv, distribution.name = "b", zaxis = T,
zlaxis = T, power = T) # Aprx

NTplot(p.hat = phat, n = n, xlim = phat + c(-3, 3)*sehat,
ylim = c(0, dnorm(phat, phat, sehat)), alpha.right = aprx.lv,
alpha.left = 0, distribution.name = "b", type = "c", zaxis = T) # Aprx

# Test and CI for One Proportion

# Small Sample Tests on a Binomial Proportion

# Test of p = 0.05 vs. p < 0.05

# Exact binomial dist:

# alt = "two.sided":

binom.test(x, n, p0,"t")

# Reject H0 if and ony if "PVal < aprx.lv".

# Reject H0 if and ony if "x < CritVal[1] or x > CritVal[2]".

(CritVal <- qbinom(c(aprx.lv/2, 1 - aprx.lv/2), n, p0))
```

```
pbinom(c(CritVal[1] - 1, CritVal[1], CritVal[2] - 1, CritVal[2]), n, p0)

(Alpha <-
1 - (pbinom(CritVal[2], n, p0) - pbinom(CritVal[1] - 1, n, p0)))

(PVal <- 2*min(1/2, pbinom(x, n, p0), 1 - pbinom(x - 1, n, p0)))

binconf(x, n, aprx.lv, "all", T, T)

1 - pbinom(x - 1, n, c(.005475565, .005475566, .005475567))

pbinom(x, n, c(.05041360, .05041361, .05041362))

(CI <- c(.005475566, .05041361))

# alt = "gr":

binom.test(x, n, p0,"g")

# Reject H0 if and ony if "PVal < aprx.lv".

# Reject H0 if and ony if "x > CritVal".

(CritVal <- qbinom(1 - aprx.lv, n, p0))

pbinom(c(CritVal - 1, CritVal), n, p0)

(Alpha <- 1 - pbinom(CritVal, n, p0))

(PVal <- 1 - pbinom(x - 1, n, p0))

binconf(x, n, 2*aprx.lv, "all", T, T)

1 - pbinom(x - 1, n, c(.006859718, .006859719, .006859720))

(CI <- c(.006859719, 1))
```

```
# alt = "less":

binom.test(x, n, p0,"l")

# Reject H0 if and ony if "PVal < aprx.lv".

# Reject H0 if and ony if "x < CritVal".

(CritVal <- qbinom(aprx.lv, n, p0))

pbinom(c(CritVal - 1, CritVal), n, p0)

(Alpha <- pbinom(CritVal - 1, n, p0))

(PVal <- pbinom(x, n, p0))

binconf(x, n, 2*aprx.lv, "all", T, T)

pbinom(x, n, c(.04518040, .04518041, .04518042))

(CI <- c(0, .04518041))

# § 4-7.2 Type II Error and Choice of Sample Size

# Aprx Spl Sz for Two- or One-Sided Alt Hypothesis on a Pop Proportion
# Using an Aprx Z-Test

AprxSplSzAprx.z.test <- function (p0, p1, aprx.lv, pwr.target, alt)
ceiling(((qnorm(1 - aprx.lv/alt)*sqrt(p0*(1 - p0)) +
qnorm(pwr.target)*sqrt(p1*(1 - p1)))/(p1 - p0))^2)

# EXAMPLE 4-13 Engine Controllers

# Refer to EXAMPLE 4-12.

# alt = "two.sided":
```

```
(AprxSplSz <- AprxSplSzAprx.z.test(p0, p1, aprx.lv, pwr.target, 2))

AprxPwrAprx.z.test(c(AprxSplSz[1] - 1, AprxSplSz[1]), p0, p1[1],
aprx.lv, 2)

AprxPwrAprx.z.test(c(AprxSplSz[2] - 1, AprxSplSz[2]), p0, p1[2],
aprx.lv, 2)

NTplot(p0 = p0, p1 = p1[1], p.hat = phat, n = AprxSplSz[1] - 1,
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz[1] - 1),
p1[1] + c(-3, 3)*sd1[1]/sqrt(AprxSplSz[1] - 1)),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz[1] - 1)),
dnorm(p1[1], p1[1], sd1[1]/sqrt(AprxSplSz[1] - 1))),
alpha.right = aprx.lv/2, alpha.left = aprx.lv/2, distribution.name = "b",
zaxis = T, zlaxis = T, power = T) # Aprx

NTplot(p0 = p0, p1 = p1[1], p.hat = phat, n = AprxSplSz[1],
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz[1]),
p1[1] + c(-3, 3)*sd1[1]/sqrt(AprxSplSz[1])),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz[1])),
dnorm(p1[1], p1[1], sd1[1]/sqrt(AprxSplSz[1]))), alpha.right = aprx.lv/2,
alpha.left = aprx.lv/2, digits = 5, distribution.name = "b", zaxis = T,
zlaxis = T, power = T) # Aprx

NTplot(p0 = p0, p1 = p1[2], p.hat = phat, n = AprxSplSz[2] - 1,
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz[2] - 1),
p1[2] + c(-3, 3)*sd1[2]/sqrt(AprxSplSz[2] - 1)),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz[2] - 1)),
dnorm(p1[2], p1[2], sd1[2]/sqrt(AprxSplSz[2] - 1))),
alpha.right = aprx.lv/2, alpha.left = aprx.lv/2, distribution.name = "b",
zaxis = T, zlaxis = T, power = T) # Aprx
```

```
NTplot(p0 = p0, p1 = p1[2], p.hat = phat, n = AprxSplSz[2],
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz[2]),
p1[2] + c(-3, 3)*sd1[2]/sqrt(AprxSplSz[2])),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz[2])),
dnorm(p1[2], p1[2], sd1[2]/sqrt(AprxSplSz[2]))), alpha.right = aprx.lv/2,
alpha.left = aprx.lv/2, distribution.name = "b", zaxis = T, zlaxis = T,
power = T) # Aprx

# alt = "gr":

(AprxSplSz <- AprxSplSzAprx.z.test(p0, p1[1], aprx.lv, pwr.target, 1))

AprxPwrAprx.z.test(c(AprxSplSz - 1, AprxSplSz), p0, p1[1], aprx.lv, 1)

NTplot(p0 = p0, p1 = p1[1], p.hat = phat, n = AprxSplSz - 1,
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz - 1),
p1[1] + c(-3, 3)*sd1[1]/sqrt(AprxSplSz - 1)),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz - 1)),
dnorm(p1[1], p1[1], sd1[1]/sqrt(AprxSplSz - 1))),
distribution.name = "b", zaxis = T, zlaxis = T, power = T) # Aprx

NTplot(p0 = p0, p1 = p1[1], p.hat = phat, n = AprxSplSz,
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz),
p1[1] + c(-3, 3)*sd1[1]/sqrt(AprxSplSz)),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz)),
dnorm(p1[1], p1[1], sd1[1]/sqrt(AprxSplSz))), distribution.name = "b",
zaxis = T, zlaxis = T, power = T) # Aprx

# alt = "less":

(AprxSplSz <- AprxSplSzAprx.z.test(p0, p1[2], aprx.lv, pwr.target, 1))

AprxPwrAprx.z.test(c(AprxSplSz - 1, AprxSplSz), p0, p1[2], aprx.lv, 1)
```

```
NTplot(p0 = p0, p1 = p1[2], p.hat = phat, n = AprxSplSz - 1,
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz - 1),
p1[2] + c(-3, 3)*sd1[2]/sqrt(AprxSplSz - 1)),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz - 1)),
dnorm(p1[2], p1[2], sd1[2]/sqrt(AprxSplSz - 1))), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", zaxis = T, zlaxis = T,
power = T) # Aprx

NTplot(p0 = p0, p1 = p1[2], p.hat = phat, n = AprxSplSz,
xlim = range(phat, p0 + c(-3, 3)*sd0/sqrt(AprxSplSz),
p1[2] + c(-3, 3)*sd1[2]/sqrt(AprxSplSz)),
ylim = range(0, dnorm(p0, p0, sd0/sqrt(AprxSplSz)),
dnorm(p1[2], p1[2], sd1[2]/sqrt(AprxSplSz))), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", zaxis = T, zlaxis = T,
power = T) # Aprx

# § 4-7.3 Confidence Interval on a Binomial Proportion

# EXAMPLE 4-14 Crankshaft Bearings

# Refer to EXAMPLE 4-12.

(n <- 85); (x <- 10); (aprx.lv <- .05); (p0 <- .05)

# Normal approx.:

(phat <- x/n); (sdhat <- sqrt(phat*(1 - phat))); (sehat <- sdhat/sqrt(n))

(a <- c(aprx.lv, aprx.lv/2))

(ptilde <- (phat + qnorm(a)^2/(2*n))/(1 + qnorm(a)^2/n))

(setilde <- sqrt(sehat^2 + qnorm(a)^2/(4*n^2))/(1 + qnorm(a)^2/n))

(sdtilde <- sqrt(n)*setilde)

# alt = "two.sided":
```

```
prop.test(x, n, p0, "t", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(c(aprx.lv/2, 1 - aprx.lv/2))) # Aprx

(AsymptoticCI <- phat + CritVal*sehat) # Aprx

(AgrestiCoullCI <- ptilde[2] + CritVal*setilde[2]) # Aprx

binconf(x, n, alpha, "all", T, T) # Aprx

NTplot(p.hat = phat, n = n, xlim = phat + c(-3, 3)*sehat,
  ylim = c(0, dnorm(phat, phat, sehat)), distribution.name = "b",
  type = "c", zaxis = T) # Aprx

NTplot(xbar = ptilde[2], n = n, sd = sdtilde[2],
  xlim = ptilde[2] + c(-3, 3)*setilde[2], alpha.right = a[2],
  alpha.left = a[2], ylim = c(0, dnorm(ptilde[2], ptilde[2], setilde[2])),
  type = "c", zaxis = T) # Aprx

# alt = "gr":

prop.test(x, n, p0, "g", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(1 - aprx.lv)) # Aprx

(AsymptoticCI <- c(phat - CritVal*sehat, 1)) # Aprx

(AgrestiCoullCI <- c(ptilde[1] - CritVal*setilde[1], 1)) # Aprx

binconf(x, n, 2*aprx.lv, "all", T, T) # Aprx

NTplot(p.hat = phat, n = n, xlim = phat + c(-3, 3)*sehat,
ylim = c(0, dnorm(phat, phat, sehat)), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", type = "c", zaxis = T)
# Aprx
```

```
NTplot(xbar = ptilde[1], n = n, sd = sdtilde[1],
  xlim = ptilde[1] + c(-3, 3)*setilde[1], alpha.right = 0,
  alpha.left = a[1], ylim = c(0, dnorm(ptilde[1], ptilde[1], setilde[1])),
  type = "c", zaxis = T) # Aprx

# alt = "less":

prop.test(x, n, p0, "l", 1 - aprx.lv, F) # Aprx

(CritVal <- qnorm(aprx.lv)) # Aprx

(AsymptoticCI <- c(0, phat - CritVal*sehat)) # Aprx

(AgrestiCoullCI <- c(0, ptilde[1] - CritVal*setilde[1])) # Aprx

binconf(x, n, 2*aprx.lv, "all", T, T) # Aprx

NTplot(p.hat = phat, n = n, xlim = phat + c(-3, 3)*sehat,
ylim = c(0, dnorm(phat, phat, sehat)), alpha.right = aprx.lv,
alpha.left = 0, distribution.name = "b", type = "c", zaxis = T) # Aprx

NTplot(xbar = ptilde[1], n = n, sd = sdtilde[1],
  xlim = ptilde[1] + c(-3, 3)*setilde[1], alpha.right = a[1],
  alpha.left = 0, ylim = c(0, dnorm(ptilde[1], ptilde[1], setilde[1])),
  type = "c", zaxis = T) # Aprx

# Choice of Sample Size

# Aprx Spl Sz for a Specified E with Two-Sided CI [phat - E, phat + E] or
# One-Sided CI [phat - E, 1) or (0, phat + E] on a Binomial Proportion
# Using Normal Aprx

AprxSplSzAprx.z.CI <- function (E, p, conf.target, alt)
ceiling((qnorm(1 - (1 - conf.target)/alt)/E)^2*p*(1 - p))

# EXAMPLE 4-15 Crankshaft Bearings
```

```
# Refer to EXAMPLE 4-14.

(conf.target <- .95); (E <- .05)

# Two-Sided CI: [phat - E, phat + E]

(AprxSplSz <- AprxSplSzAprx.z.CI(E, phat, conf.target, 2))

NTplot(p.hat = phat, n = AprxSplSz - 1,
xlim = phat + c(-3, 3)*sdhat/sqrt(AprxSplSz - 1),
ylim = c(0, dnorm(phat, phat, sdhat/sqrt(AprxSplSz - 1))), digits = 5,
distribution.name = "b", type = "c", zaxis = T) # Aprx

NTplot(p.hat = phat, n = AprxSplSz,
xlim = phat + c(-3, 3)*sdhat/sqrt(AprxSplSz),
ylim = c(0, dnorm(phat, phat, sdhat/sqrt(AprxSplSz))), digits = 5,
distribution.name = "b", type = "c", zaxis = T) # Aprx

(AprxSplSz <- AprxSplSzAprx.z.CI(E, 1/2, conf.target, 2))

NTplot(p.hat = 1/2, n = AprxSplSz - 1,
xlim = 1/2 + c(-3, 3)*(1/2)/sqrt(AprxSplSz - 1),
ylim = c(0, dnorm(1/2, 1/2, (1/2)/sqrt(AprxSplSz - 1))), digits = 5,
distribution.name = "b", type = "c", zaxis = T) # Aprx

NTplot(p.hat = 1/2, n = AprxSplSz,
xlim = 1/2 + c(-3, 3)*(1/2)/sqrt(AprxSplSz),
ylim = c(0, dnorm(1/2, 1/2, (1/2)/sqrt(AprxSplSz))), digits = 5,
distribution.name = "b", type = "c", zaxis = T) # Aprx

# One-Sided CI: [phat - E, 1) or (0, phat + E]

(AprxSplSz <- AprxSplSzAprx.z.CI(E, phat, conf.target, 1))
```

```
NTplot(p.hat = phat, n = AprxSplSz - 1,
xlim = phat + c(-3, 3)*sdhat/sqrt(AprxSplSz - 1),
ylim = c(0, dnorm(phat, phat, sdhat/sqrt(AprxSplSz - 1))), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", type = "c", zaxis = T)
# Aprx

NTplot(p.hat = phat, n = AprxSplSz,
xlim = phat + c(-3, 3)*sdhat/sqrt(AprxSplSz),
ylim = c(0, dnorm(phat, phat, sdhat/sqrt(AprxSplSz))), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", type = "c", zaxis = T)
# Aprx

NTplot(p.hat = phat, n = AprxSplSz - 1,
xlim = phat + c(-3, 3)*sdhat/sqrt(AprxSplSz - 1),
ylim = c(0, dnorm(phat, phat, sdhat/sqrt(AprxSplSz - 1))),
alpha.right = aprx.lv, alpha.left = 0, digits = 5, distribution.name = "b",
type = "c", zaxis = T) # Aprx

NTplot(p.hat = phat, n = AprxSplSz,
xlim = phat + c(-3, 3)*sdhat/sqrt(AprxSplSz),
ylim = c(0, dnorm(phat, phat, sdhat/sqrt(AprxSplSz))),
alpha.right = aprx.lv, alpha.left = 0, distribution.name = "b", type = "c",
zaxis = T) # Aprx

(AprxSplSz <- AprxSplSzAprx.z.CI(E, 1/2, conf.target, 1))

NTplot(p.hat = 1/2, n = AprxSplSz - 1,
xlim = 1/2 + c(-3, 3)*(1/2)/sqrt(AprxSplSz - 1),
ylim = c(0, dnorm(1/2, 1/2, (1/2)/sqrt(AprxSplSz - 1))), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", type = "c", zaxis = T)
# Aprx

NTplot(p.hat = 1/2, n = AprxSplSz,
xlim = 1/2 + c(-3, 3)*(1/2)/sqrt(AprxSplSz),
ylim = c(0, dnorm(1/2, 1/2, (1/2)/sqrt(AprxSplSz))), alpha.right = 0,
alpha.left = aprx.lv, distribution.name = "b", type = "c", zaxis = T)
# Aprx
```

```
NTplot(p.hat = 1/2, n = AprxSplSz - 1,
xlim = 1/2 + c(-3, 3)*(1/2)/sqrt(AprxSplSz - 1),
ylim = c(0, dnorm(1/2, 1/2, (1/2)/sqrt(AprxSplSz - 1))),
alpha.right = aprx.lv, alpha.left = 0, digits = 5, distribution.name = "b",
type = "c", zaxis = T) # Aprx

NTplot(p.hat = 1/2, n = AprxSplSz,
xlim = 1/2 + c(-3, 3)*(1/2)/sqrt(AprxSplSz),
ylim = c(0, dnorm(1/2, 1/2, (1/2)/sqrt(AprxSplSz))),
alpha.right = aprx.lv, alpha.left = 0, distribution.name = "b", type = "c",
zaxis = T) # Aprx

# EXAMPLE 4-16 The Agresti-Coull CI on a Proportion

# Refer to EXAMPLE 4-14.

# EXERCISES FOR § 4-7
```

```
# § 4-8 Other Interval Estimates for a Single Sample

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(HH, tolerance)

# § 4-8.1 Prediction Interval

# EXAMPLE 4-17 Golf Clubs

# Refer to EXAMPLE 4-7.

(n <- length(COR)); (xbar <- mean(COR)); (s <- sd(COR))

(df <- length(COR) - 1); (alpha <- .05); (s.pred <- s*sqrt(n + 1))

(se.pred <- s.pred/sqrt(n))

(COR.pred <- xbar + s.pred*(COR - xbar)/s)

(mean(COR.pred)); (sd(COR.pred))

# Two-Sided PI: [LPL, UPL]

(CritVal <- qt(c(alpha/2, 1 - alpha/2), df))

(PI <- xbar + CritVal*se.pred)

NTplot(t.test(COR.pred, NULL, "t", mu0), "c", zaxis = T)

# One-Sided PI: [LPL, Inf)

(CritVal <- qt(alpha, df))

(PI <- c(xbar + CritVal*se.pred, Inf))

NTplot(t.test(COR.pred, NULL, "g", mu0), "c", zaxis = T)
```

```
# One-Sided PI: (-Inf, UPL]

(CritVal <- qt(1 - alpha, df))

(PI <- c(-Inf, xbar + CritVal*se.pred))

NTplot(t.test(COR.pred, NULL, "l", mu0), "c", zaxis = T)

# § 4-8.2 Tolerance Intervals for a Normal Distribution

# EXAMPLE 4-18 Golf Clubs

# Refer to EXAMPLE 4-7.

(covrg.p.at.least <- .95); (conf.lv <- .9)

# Tables of K-factors for TIs Based on Normality

# Two-Sided:

K.table(n, c(.01, .05, .1), c(.9, .95, .99), 2, NULL, "EX", 50, "n")

(tol.out2 <- normtol.int(COR, 1 - conf.lv, covrg.p.at.least, 2, "EX", 50))

plottol(tol.out2, COR, NULL, NULL, "t", "b", x.lab = "COR")

# One-Sided:

K.table(n, c(.01, .05, .1), c(.9, .95, .99), 1, NULL, "EX", 50, "n")

(tol.out1 <- normtol.int(COR, 1 - conf.lv, covrg.p.at.least, 1, "EX", 50))

plottol(tol.out1, COR, NULL, NULL, "l", "b", x.lab = "COR")

plottol(tol.out1, COR, NULL, NULL, "u", "b", x.lab = "COR")

# EXERCISES FOR § 4-8
```

```
# § 4-9 Summary Tables of Inference Procedures for a Single Sample

# § 4-10 Testing for Goodness of Fit

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(vcd)

# EXAMPLE 4-19 Printed Circuit Boards

(No.Defects <- sample(c(rep(0, 32), rep(1, 15), rep(2, 9), rep(3, 4))))

table(No.Defects); print(goodfit(No.Defects), "d")

summary(goodfit(No.Defects)); plot(goodfit(No.Defects))

(No.Defects.Comb. <- sample(c(rep(0, 32), rep(1, 15), rep(2, 13))))

table(No.Defects.Comb.); print(goodfit(No.Defects.Comb.))

summary(goodfit(No.Defects.Comb., "p", "Mi"))

plot(goodfit(No.Defects.Comb., "p", "Mi"))

# EXERCISES FOR § 4-10
```